

Informe de cierre del sistema de donantes

Planificación implementación y mejora continua

Cierre verificado el 30 de septiembre de 2026

Se completó el registro de donantes con una interfaz web minimalista, SQLite, autenticación JWT y roles de administrador y usuario. Las pruebas, SonarQube y OWASP ZAP se ejecutaron realmente. GitHub Actions construyó y desplegó el contenedor de prueba, verificó su funcionamiento y terminó correctamente.

Planificación frente a implementación

Etapa	Plan	Resultado comprobado
Implementación y seguridad	4 horas	API persistente, contraseñas scrypt, permisos y despliegue automático en Docker.
Pruebas y calidad	3 horas	9 pruebas aprobadas, cobertura 100 %, auditoría, SonarQube y ZAP completados.
Cierre y evaluación	2 horas	Repositorio Git, reportes reales, documento de cierre y ZIP con manifiesto.

Las nueve horas corresponden al presupuesto propuesto, no a horas reales registradas. No se dispone de un registro completo para calcular desviaciones de esfuerzo o retrasos en días.

Desviaciones y resolución

La primera entrega dejó los análisis externos pendientes por falta de herramientas y acceso configurado. Para el cierre se descargaron versiones portátiles, se resolvió un problema de sockets de Java en Windows y se preparó un pipeline autónomo con SonarQube temporal.

El primer intento de GitHub Actions falló al iniciar ZAP por una configuración de alcance que excluía la raíz del destino. Se corrigió el alcance y se añadió una URL inicial al árbol de escaneo. La siguiente ejecución completó todos los pasos, incluido el despliegue y la limpieza.

La interfaz permite crear cuentas, iniciar sesión, buscar y registrar donantes; el administrador puede eliminarlos con confirmación. Se verificó el flujo en Chromium y la vista móvil de 390 px. El entorno de prueba se creó dentro del runner y se eliminó al finalizar; no se ofrece un servidor público permanente ni un historial de donaciones.

Resultados y métricas finales

Indicador	Resultado
Jest	9 pruebas aprobadas de 9; una suite aprobada
Cobertura de líneas sentencias ramas y funciones	100 %; umbral exigido de 80 %
Auditoría de dependencias de producción	0 vulnerabilidades conocidas informadas
SonarQube bugs vulnerabilidades y hotspots	0 en cada métrica
SonarQube code smells y deuda técnica	0 observaciones; 0 minutos de deuda
SonarQube cobertura y duplicación	100 % importado; 0 % de duplicación
Quality gate y GitHub Actions	OK y success respectivamente

La cobertura instrumentada comprende src/app.js y src/store.js. La interfaz, el arranque y la utilidad administrativa quedan fuera del porcentaje de cobertura. SonarQube analiza src; los scripts auxiliares de CI no forman parte de ese alcance. El análisis inicial señaló dos code smells y 10 minutos de deuda: ambos se corrigieron y la nueva ejecución confirmó cero.

Escaneo de seguridad con ZAP

Rol	Altas	Medias	Bajas	Informativas	Peticiones activas
Usuario	0	0	0	24	827
Administrador	0	0	0	24	802

Ambos escaneos finalizaron al 100 % y se vació la cola pasiva antes de exportar. Las reglas XSS reflejado y SQLi finalizaron sin alertas. Las entradas de reglas OAST y scripts personalizados no configurados constan como omitidas; el detalle está en el reporte de ejecución.

Las 24 instancias informativas por rol corresponden a User Agent Fuzzer. La revisión reprodujo respuestas 201/409 al repetir un correo y 204/404 al repetir un borrado, independientemente de tres User-Agent distintos. Se conservan las observaciones y su explicación; los reportes HTML agrupan algunas instancias.

El escaneo cubre salud y rutas de donantes con ambos roles. Registro y login se usan en la preparación y cuentan con pruebas de abuso, pero no con fuzzing exhaustivo. La interfaz usa CSP, recursos locales y datos insertados como texto; su prueba funcional en navegador se documenta por separado. Estos resultados acreditan el alcance comprobado, no ausencia absoluta de vulnerabilidades.

Lecciones de la planificación

Comprobar herramientas y acceso antes de estimar

La disponibilidad de Java, ZAP, SonarQube, recursos del equipo y permisos en GitHub condiciona la entrega. El plan debe incluir una comprobación temprana y tiempo para resolver problemas de instalación o compatibilidad.

Definir evidencias observables de aceptación

La planificación debe especificar rutas, roles, alcance de cobertura, métricas y artefactos. Un archivo de pipeline preparado no acredita despliegue: se necesita una ejecución completa enlazada a un commit.

Registrar esfuerzo y bloqueos

El presupuesto de nueve horas organiza el trabajo, pero no permite afirmar cuánto se tardó. En siguientes iteraciones conviene registrar inicio, fin y bloqueos por actividad para comparar el plan con datos reales.

Lecciones de la implementación

Verificar permisos desde los casos negativos

El ejemplo inicial aceptaba el rol enviado por el cliente y no verificaba contraseña. La solución recupera el rol de SQLite y prueba intentos de suplantación y acceso denegado. Separar aplicación, almacenamiento y arranque facilita aislar estos casos.

Analizar los resultados según el estado de los datos

Las solicitudes repetidas pueden modificar o eliminar datos. Las diferencias legítimas de respuesta generaron observaciones informativas en ZAP; se revisaron con distintos User-Agent y se preservaron como evidencia en vez de ocultarlas.

Ejecutar el pipeline durante el desarrollo

La primera ejecución reveló un fallo de alcance en ZAP que Jest no detectaba. Corregirlo y repetir toda la cadena permitió demostrar la entrega continua. El lockfile, los contenedores y los artefactos enlazados al commit hacen reproducible la evaluación.

Propuesta de mejora continua

Los siguientes plazos y responsables son propuestas para una siguiente iteración. La implementación y los análisis solicitados están completos; las acciones siguientes amplían la preparación para un uso permanente y la cobertura de escenarios.

Prioridad y plazo	Acción y responsable	Criterio de éxito
P1 Semana 1	HTTPS, secretos, respaldo y auditoría. Backend y DevOps.	Restauración ensayada y operaciones sensibles trazables.
P1 Semana 1	Ampliar DAST a registro/login y OAST aplicable. Seguridad.	Matriz de rutas y reglas sin huecos no justificados.
P1 Semana 2	Hashing asíncrono, límites compartidos y carga. Backend.	Latencia y concurrencia objetivo definidas y medidas.
P2 Semana 2	Recuperación de cuentas y revocación de tokens. Backend.	Pruebas funcionales y de abuso aprobadas.
P2 Quincenal	Actualizar dependencias y revisar métricas. Calidad.	Cobertura $\geq 80\%$ y hallazgos altos tratados.
P3 Siguiete iteración	Explorar predicción de donaciones. Datos y producto.	Historial suficiente, línea base y error medido.

La propuesta de inteligencia artificial requiere incorporar historial de donaciones y una base de uso apropiada de los datos. El registro de personas por sí solo no permite validar predicciones. Esta funcionalidad no se declara implementada.

Trazabilidad de la entrega

Commit de implementación analizado: `e4ba12e80748d1cf6f0c6d05af330f079c675e4d`

Repositorio: [GuillermoJC456/Proyecto-FINALGECJ](#)

Ejecución aprobada: [GitHub Actions 36756949259](#)

Los reportes están en `reports/unitarias`, `reports/seguridad` y `reports/sonarqube`; los metadatos y logs de CI en `reports/ci`. El ZIP conserva `.git`, historial y manifiesto SHA256. La actualización documental posterior no cambia el código evaluado.